# Recitation 5 Accelerometer & Gyroscope

## Agenda

1. Introduction to LSM6DSL sensor
2. How to read data from the LSM6DSL sensor using polling
3. Interrupts — software and hardware
4. How to use Teleplot

## LSM6DSL Sensor

### Sensor Ranges and Configuration

The LSM6DSL is a versatile 6-axis inertial measurement unit (IMU) that includes an accelerometer and gyroscope. One of its key features is the ability to operate at different measurement ranges to accommodate various applications.

**Available Measurement Ranges**

**Accelerometer Ranges: [acceleration due to gravity]**

- ±2g
- ±4g
- ±8g
- ±16g

**Gyroscope Ranges: [dps = degrees per second (angular velocity)]**

- ±125 dps
- ±250 dps
- ±500 dps
- ±1000 dps
- ±2000 dps

### Accelerometer Range Configuration

Bits [3:2] in the CTRL1_XL register control the accelerometer's full-scale range:

- 00 = ±2g
- 10 = ±4g
- 11 = ±8g
- 01 = ±16g

```
// 1. For ±2g range (default)
write_register(CTRL1_XL, 0x40);  // 0100 0000: ODR=104Hz, FS=±2g
const float ACC_SENSITIVITY = 0.061f;  // mg/LSB

// 2. For ±4g range
```

```
write_register(CTRL1_XL, 0x48);  // 0100 1000: ODR=104Hz, FS=±4g
const float ACC_SENSITIVITY = 0.122f;  // mg/LSB

// 3. For ±8g range
write_register(CTRL1_XL, 0x4C);  // 0100 1100: ODR=104Hz, FS=±8g
const float ACC_SENSITIVITY = 0.244f;  // mg/LSB

// 4. For ±16g range
write_register(CTRL1_XL, 0x44);  // 0100 0100: ODR=104Hz, FS=±16g
const float ACC_SENSITIVITY = 0.488f;  // mg/LSB
```

Example code to set ±8g range:

```
// Configure accelerometer for ±8g range
write_register(CTRL1_XL, 0x4C);
printf("Accelerometer configured: 104 Hz, ±8g range\r\n");

// Use appropriate sensitivity for ±8g range
const float ACC_SENSITIVITY = 0.244f;  // mg/LSB
```

### Gyroscope Range Configuration

Bits [3:2] in the CTRL2_G register control the gyroscope's full-scale range:

- 00 = ±250 dps
- 01 = ±500 dps
- 10 = ±1000 dps
- 11 = ±2000 dps

Note: ±125 dps uses a different bit configuration (bit 1 = 1)

```
// 1. For ±125 dps range
write_register(CTRL2_G, 0x42);  // 0100 0010: ODR=104Hz, FS=±125dps
const float GYRO_SENSITIVITY = 4.375f;  // mdps/LSB

// 2. For ±250 dps range (default)
write_register(CTRL2_G, 0x40);  // 0100 0000: ODR=104Hz, FS=±250dps
const float GYRO_SENSITIVITY = 8.75f;   // mdps/LSB

// 3. For ±500 dps range
write_register(CTRL2_G, 0x44);  // 0100 0100: ODR=104Hz, FS=±500dps
const float GYRO_SENSITIVITY = 17.5f;   // mdps/LSB

// 4. For ±1000 dps range
write_register(CTRL2_G, 0x48);  // 0100 1000: ODR=104Hz, FS=±1000dps
const float GYRO_SENSITIVITY = 35.0f;   // mdps/LSB

// 5. For ±2000 dps range
```

```
write_register(CTRL2_G, 0x4C);  // 0100 1100: ODR=104Hz, FS=±2000dps
const float GYRO_SENSITIVITY = 70.0f;   // mdps/LSB
```

Example code to set ±1000 dps range:

```
// Configure gyroscope for ±1000 dps range
write_register(CTRL2_G, 0x48);
printf("Gyroscope configured: 104 Hz, ±1000 dps range\r\n");

// Use appropriate sensitivity for ±1000 dps range
const float GYRO_SENSITIVITY = 35.0f;  // mdps/LSB
```

### ODR (Output Data Rate)

ODR stands for Output Data Rate — it defines how many samples per second the sensor produces. In this recitation we use 104Hz, meaning the sensor generates a new accelerometer and gyroscope reading approximately every 9.6ms. The ODR is set in the upper 4 bits of CTRL1_XL and CTRL2_G registers alongside the range configuration.

### Raw Values and Sensitivity

The sensor outputs a 16-bit signed integer (range: -32768 to +32767) for each axis. This raw number has no physical meaning on its own — it just represents a voltage level from the sensor's internal analog circuit. To convert it to a real physical unit (g for acceleration, dps for rotation) we multiply by the sensitivity factor, which depends on the measurement range configured:

```
actual_value [g]   = raw_value × sensitivity [mg/LSB] / 1000
actual_value [dps] = raw_value × sensitivity [mdps/LSB] / 1000
```

The larger the range (e.g. ±16g vs ±2g), the less precise each step is — each LSB represents a bigger physical change. This is the trade-off between range and resolution.

### Little-Endian Byte Order

Each axis value is 16 bits wide but I2C transfers only 8 bits at a time, so the sensor splits each reading across two registers: a low byte and a high byte. The LSM6DSL uses little-endian order, meaning the low byte comes first. We read both bytes separately and then combine them:

```
int16_t read_16bit_value(uint8_t low_reg, uint8_t high_reg) {
    char low_byte  = read_register(low_reg);   // Read low byte first
    char high_byte = read_register(high_reg);  // Then high byte
    return (high_byte << 8) | low_byte;        // Shift high byte left 8
bits, OR with low byte
}
```

For example if low byte = 0x34 and high byte = 0x12, the result is 0x1234.

### WHO_AM_I Register

WHO_AM_I (register 0x0F) is a read-only register built into the LSM6DSL that always returns the fixed value 0x6A. It exists purely for identification — we read it at startup to confirm that the sensor is connected, powered, and responding correctly over I2C before we try to configure or read it. If we get back anything other than 0x6A, something is wrong with the wiring or the address.

```
uint8_t id = read_register(WHO_AM_I);
printf("WHO_AM_I = 0x%02X (Expected: 0x6A)\r\n", id);

if (id != 0x6A) {
    printf("Error: LSM6DSL sensor not found!\r\n");
    while (1) {}  // Stop here, do not continue
}
```

## Polling

Polling means the microcontroller checks the sensor on a fixed timer, regardless of whether new data is actually ready. In this example the main loop wakes up every 200ms, reads all axes, and prints the values. The sensor produces data at 104Hz (~every 9.6ms) but we only read every 200ms — most samples are simply ignored. It is simple to implement but inefficient compared to interrupt-driven approaches.

### Example — Polling

```
#include "mbed.h"

I2C i2c(PB_11, PB_10); // I2C2: SDA = PB11, SCL = PB10

// Ignore this — sometimes Mac needs this to properly use printf
BufferedSerial serial_port(USBTX, USBRX, 115200);
FileHandle *mbed::mbed_override_console(int) { return &serial_port; }

#define LSM6DSL_ADDR (0x6A << 1)
#define WHO_AM_I  0x0F
#define CTRL1_XL  0x10
#define CTRL2_G   0x11
#define OUTX_L_XL 0x28
#define OUTX_H_XL 0x29
#define OUTY_L_XL 0x2A
#define OUTY_H_XL 0x2B
#define OUTZ_L_XL 0x2C
#define OUTZ_H_XL 0x2D
#define OUTX_L_G  0x22
#define OUTX_H_G  0x23
#define OUTY_L_G  0x24
#define OUTY_H_G  0x25
#define OUTZ_L_G  0x26
```

```
#define OUTZ_H_G  0x27

void write_register(uint8_t reg, uint8_t value) {
    char data[2] = {(char)reg, (char)value};
    i2c.write(LSM6DSL_ADDR, data, 2);
}

uint8_t read_register(uint8_t reg) {
    char data = reg;
    i2c.write(LSM6DSL_ADDR, &data, 1, true); // No stop
    i2c.read(LSM6DSL_ADDR, &data, 1);
    return (uint8_t)data;
}

int16_t read_16bit_value(uint8_t low_reg, uint8_t high_reg) {
    char low_byte  = read_register(low_reg);
    char high_byte = read_register(high_reg);
    return (high_byte << 8) | low_byte; // little-endian
}

int main() {
    i2c.frequency(400000);

    uint8_t id = read_register(WHO_AM_I);
    printf("WHO_AM_I = 0x%02X (Expected: 0x6A)\r\n", id);

    if (id != 0x6A) {
        printf("Error: LSM6DSL sensor not found!\r\n");
        while (1) {}
    }

    // Configure accelerometer — change register value and sensitivity to
match your chosen range
    write_register(CTRL1_XL, 0x40);          // 104Hz, ±2g
    const float ACC_SENSITIVITY  = 0.061f;   // mg/LSB for ±2g

    // Configure gyroscope — change register value and sensitivity to
match your chosen range
    write_register(CTRL2_G, 0x40);           // 104Hz, ±250 dps
    const float GYRO_SENSITIVITY = 8.75f;    // mdps/LSB for ±250 dps

    while (1) {
        int16_t acc_x_raw = read_16bit_value(OUTX_L_XL, OUTX_H_XL);
        int16_t acc_y_raw = read_16bit_value(OUTY_L_XL, OUTY_H_XL);
        int16_t acc_z_raw = read_16bit_value(OUTZ_L_XL, OUTZ_H_XL);

        int16_t gyro_x_raw = read_16bit_value(OUTX_L_G, OUTX_H_G);
        int16_t gyro_y_raw = read_16bit_value(OUTY_L_G, OUTY_H_G);
        int16_t gyro_z_raw = read_16bit_value(OUTZ_L_G, OUTZ_H_G);

        // Convert to g (sensitivity is in mg/LSB, divide by 1000 to get
g)
        float acc_x_g = acc_x_raw * ACC_SENSITIVITY / 1000.0f;
        float acc_y_g = acc_y_raw * ACC_SENSITIVITY / 1000.0f;
```

```
        float acc_z_g = acc_z_raw * ACC_SENSITIVITY / 1000.0f;

        // Convert to dps (sensitivity is in mdps/LSB, divide by 1000 to
get dps)
        float gyro_x_dps = gyro_x_raw * GYRO_SENSITIVITY / 1000.0f;
        float gyro_y_dps = gyro_y_raw * GYRO_SENSITIVITY / 1000.0f;
        float gyro_z_dps = gyro_z_raw * GYRO_SENSITIVITY / 1000.0f;

        // Human-readable print
        printf("Accel [g]: X=%+6.3f, Y=%+6.3f, Z=%+6.3f | Gyro [dps]:
X=%+7.2f, Y=%+7.2f, Z=%+7.2f\r\n",
               acc_x_g, acc_y_g, acc_z_g, gyro_x_dps, gyro_y_dps,
gyro_z_dps);

        // Teleplot format
        printf(">acc_x:%.3f\n>acc_y:%.3f\n>acc_z:%.3f\n"
               ">gyro_x:%.2f\n>gyro_y:%.2f\n>gyro_z:%.2f\n",
               acc_x_g, acc_y_g, acc_z_g, gyro_x_dps, gyro_y_dps,
gyro_z_dps);

        // Polling delay - wakes up every 200ms regardless of sensor data
rate
        ThisThread::sleep_for(200ms);
    }
}
```

## Interrupts

### What is an Interrupt?

An interrupt is a signal that tells the microcontroller to stop what it is currently doing and immediately run a specific function called an **Interrupt Service Routine (ISR)**. Once the ISR finishes, the microcontroller goes back to what it was doing before. This is much more efficient than polling because the CPU is not wasting time constantly checking — it only reacts when something actually happens.

### Software Interrupts vs Hardware Interrupts

**Software interrupts** are triggered by the program itself — for example a timer that fires every 1ms, or a watchdog timeout. They are generated internally by the microcontroller's own peripherals.

**Hardware interrupts** are triggered by an external physical signal on a GPIO pin. For example, pressing a button or a sensor asserting a data-ready pin. The signal comes from outside the chip and causes the CPU to immediately jump to the ISR.

In our case the LSM6DSL has a dedicated **INT1 pin** (connected to PD_11 on the STM32L475E-IOT01A). Every time the sensor has a new sample ready, it drives this pin HIGH for ~50µs. We configure the MCU to detect this rising edge and call our ISR instantly — no polling needed.

### ISR Rules — What You Must and Must Not Do

ISRs run with very high priority and interrupt the normal program flow. Because of this there are strict rules:

- **Keep ISRs as short as possible** — just set a flag and return. The less time spent in the ISR the better.
- **Never call printf inside an ISR** — printf uses locks and buffers that are not interrupt-safe and will cause a crash or hang.
- **Never do I2C or SPI reads inside an ISR** — these are blocking operations and will deadlock.
- **Always declare shared variables as `volatile`** — this tells the compiler the variable can change at any time outside normal program flow, preventing incorrect optimizations.

```
// CORRECT — ISR just sets a flag
volatile bool data_ready = false;

void data_ready_isr() {
    data_ready = true;  // fast, safe
}

// WRONG — never do this inside an ISR
void bad_isr() {
    printf("data ready!\n");        // CRASH
    read_register(WHO_AM_I);        // DEADLOCK
}
```

### New Registers Used in Interrupt Mode

Compared to polling, the interrupt example uses three additional registers:

- **CTRL3_C (0x12)** — set to `0x44` to enable Block Data Update (BDU) and auto-increment. BDU prevents the sensor updating its output registers mid-read, so you never get a mismatched high/low byte pair.
- **INT1_CTRL (0x0D)** — set to `0x03` to route both accelerometer and gyroscope data-ready signals to the INT1 pin.
- **DRDY_PULSE_CFG (0x0B)** — set to `0x80` to make the INT1 pin fire a short 50µs pulse instead of holding HIGH until data is read. This works better with `InterruptIn` on Mbed.

### Example — Interrupt Driven (Single Thread)

```
#include "mbed.h"

I2C i2c(PB_11, PB_10);

#define LSM6DSL_ADDR   (0x6A << 1)
#define WHO_AM_I       0x0F
#define CTRL1_XL       0x10
#define CTRL2_G        0x11
#define CTRL3_C        0x12
#define DRDY_PULSE_CFG 0x0B
#define INT1_CTRL      0x0D
#define STATUS_REG     0x1E
#define OUTX_L_G       0x22
#define OUTX_L_XL      0x28
```

```
// Hardware interrupt on INT1 pin — fires when sensor has new data
InterruptIn int1(PD_11, PullDown);

// Flag set by ISR — must be volatile so compiler does not optimize it
away
volatile bool data_ready = false;

// ISR — keep it tiny, just set the flag
void data_ready_isr() {
    data_ready = true;
}

bool write_reg(uint8_t reg, uint8_t val) {
    char buf[2] = {(char)reg, (char)val};
    return (i2c.write(LSM6DSL_ADDR, buf, 2) == 0);
}

bool read_reg(uint8_t reg, uint8_t &val) {
    char r = (char)reg;
    if (i2c.write(LSM6DSL_ADDR, &r, 1, true) != 0) return false;
    if (i2c.read(LSM6DSL_ADDR, &r, 1) != 0) return false;
    val = (uint8_t)r;
    return true;
}

bool read_int16(uint8_t reg_low, int16_t &val) {
    uint8_t lo, hi;
    if (!read_reg(reg_low, lo)) return false;
    if (!read_reg(reg_low + 1, hi)) return false;
    val = (int16_t)((hi << 8) | lo);
    return true;
}

bool init_sensor() {
    uint8_t who;
    if (!read_reg(WHO_AM_I, who) || who != 0x6A) {
        printf("Sensor not found!\r\n");
        return false;
    }

    write_reg(CTRL3_C, 0x44);         // Enable BDU + auto-increment
    write_reg(CTRL1_XL, 0x44);        // 104Hz, ±16g
    write_reg(CTRL2_G, 0x40);         // 104Hz, ±250 dps
    write_reg(INT1_CTRL, 0x03);       // Route data-ready to INT1 pin
    write_reg(DRDY_PULSE_CFG, 0x80); // Use 50us pulse on INT1

    ThisThread::sleep_for(100ms);

    // Clear any stale data
    uint8_t dummy;
    read_reg(STATUS_REG, dummy);
    int16_t temp;
    for (int i = 0; i < 6; i++) {
```

```
            read_int16(OUTX_L_XL + i*2, temp);
        }

        // Attach ISR to rising edge of INT1 pin
        int1.rise(&data_ready_isr);

        return true;
    }

    void read_sensor_data() {
        int16_t acc[3], gyro[3];

        for (int i = 0; i < 3; i++) {
            read_int16(OUTX_L_XL + i*2, acc[i]);
            read_int16(OUTX_L_G  + i*2, gyro[i]);
        }

        // ±16g: 0.488 mg/LSB → divide by 1000 for g
        float ax = acc[0] * 0.000488f;
        float ay = acc[1] * 0.000488f;
        float az = acc[2] * 0.000488f;

        // ±250 dps: 8.75 mdps/LSB → divide by 1000 for dps
        float gx = gyro[0] * 0.00875f;
        float gy = gyro[1] * 0.00875f;
        float gz = gyro[2] * 0.00875f;

        printf(">acc_x:%.3f\n>acc_y:%.3f\n>acc_z:%.3f\n"
               ">gyro_x:%.2f\n>gyro_y:%.2f\n>gyro_z:%.2f\n",
               ax, ay, az, gx, gy, gz);
    }

    int main() {
        static BufferedSerial pc(USBTX, USBRX, 115200);
        i2c.frequency(400000);

        if (!init_sensor()) {
            while(1) { ThisThread::sleep_for(1s); }
        }

        while (true) {
            if (data_ready) {
                data_ready = false;     // Clear flag before reading
                read_sensor_data();     // Now safe to read I2C
            }
            ThisThread::sleep_for(1ms); // Prevent busy-waiting
        }
    }
```

Example — Interrupt Driven with Threads and EventQueue

When the sensor fires at 104Hz, reading I2C and printing to serial both happen in the same main loop. If `printf` is slow it can delay the next read. The solution is to split the work into two threads — one dedicated to reading the sensor, one dedicated to printing — and pass data between them using an **EventQueue**.

The EventQueue acts as a safe buffer: the acquisition thread posts a print job onto the queue, and the print thread processes it whenever it gets CPU time. This way the two operations never block each other.

```
#include "mbed.h"

I2C i2c(PB_11, PB_10);

#define LSM6DSL_ADDR   (0x6A << 1)
#define WHO_AM_I       0x0F
#define CTRL1_XL       0x10
#define CTRL2_G        0x11
#define CTRL3_C        0x12
#define DRDY_PULSE_CFG 0x0B
#define INT1_CTRL      0x0D
#define STATUS_REG     0x1E
#define OUTX_L_G       0x22
#define OUTX_L_XL      0x28

// Struct to package one complete IMU sample for passing between threads
typedef struct {
    float acc[3];
    float gyro[3];
} ImuSample;

// EventQueue used to safely post print jobs from acquisition thread to
print thread
EventQueue print_queue;

InterruptIn int1(PD_11, PullDown);
volatile bool data_ready = false;

void data_ready_isr() {
    data_ready = true;
}

bool write_reg(uint8_t reg, uint8_t val) {
    char buf[2] = {(char)reg, (char)val};
    return (i2c.write(LSM6DSL_ADDR, buf, 2) == 0);
}

bool read_reg(uint8_t reg, uint8_t &val) {
    char r = (char)reg;
    if (i2c.write(LSM6DSL_ADDR, &r, 1, true) != 0) return false;
    if (i2c.read(LSM6DSL_ADDR, &r, 1) != 0) return false;
    val = (uint8_t)r;
    return true;
}
```

```
bool read_int16(uint8_t reg_low, int16_t &val) {
    uint8_t lo, hi;
    if (!read_reg(reg_low, lo)) return false;
    if (!read_reg(reg_low + 1, hi)) return false;
    val = (int16_t)((hi << 8) | lo);
    return true;
}

bool init_sensor() {
    uint8_t who;
    if (!read_reg(WHO_AM_I, who) || who != 0x6A) {
        printf("Sensor not found!\r\n");
        return false;
    }

    write_reg(CTRL3_C, 0x44);
    write_reg(CTRL1_XL, 0x44);       // 104Hz, ±16g
    write_reg(CTRL2_G, 0x40);        // 104Hz, ±250 dps
    write_reg(INT1_CTRL, 0x03);
    write_reg(DRDY_PULSE_CFG, 0x80);

    ThisThread::sleep_for(100ms);

    uint8_t dummy;
    read_reg(STATUS_REG, dummy);
    int16_t temp;
    for (int i = 0; i < 6; i++) {
        read_int16(OUTX_L_XL + i*2, temp);
    }

    int1.rise(&data_ready_isr);
    return true;
}

// Called by the print thread via the event queue — safe to printf here
void print_sample(ImuSample sample) {
    printf(">acc_x:%.3f\n>acc_y:%.3f\n>acc_z:%.3f\n"
           ">gyro_x:%.2f\n>gyro_y:%.2f\n>gyro_z:%.2f\n",
           sample.acc[0], sample.acc[1], sample.acc[2],
           sample.gyro[0], sample.gyro[1], sample.gyro[2]);
}

// Reads sensor and posts a print job to the queue — does NOT print
directly
void read_sensor_data() {
    int16_t acc[3], gyro[3];

    for (int i = 0; i < 3; i++) {
        read_int16(OUTX_L_XL + i*2, acc[i]);
        read_int16(OUTX_L_G  + i*2, gyro[i]);
    }

    ImuSample sample;
    sample.acc[0]  = acc[0]  * 0.000488f;
```

```
        sample.acc[1]  = acc[1]  * 0.000488f;
        sample.acc[2]  = acc[2]  * 0.000488f;
        sample.gyro[0] = gyro[0] * 0.00875f;
        sample.gyro[1] = gyro[1] * 0.00875f;
        sample.gyro[2] = gyro[2] * 0.00875f;

        // Post to queue — print_sample will be called by the print thread
        print_queue.call(print_sample, sample);
    }

    // Acquisition thread — waits for ISR flag and reads sensor
    void acquisition_task() {
        while (true) {
            if (data_ready) {
                data_ready = false;
                read_sensor_data();
            }
            ThisThread::sleep_for(1ms);
        }
    }

    // Print thread — processes the event queue forever
    void print_task() {
        print_queue.dispatch_forever();
    }

    int main() {
        static BufferedSerial pc(USBTX, USBRX, 115200);
        i2c.frequency(400000);

        if (!init_sensor()) {
            while(1) { ThisThread::sleep_for(1s); }
        }

        // Start acquisition and print threads
        Thread acq_thread;
        acq_thread.start(acquisition_task);

        Thread print_thread;
        print_thread.start(print_task);

        // Main thread is idle — all work happens in the two threads above
        while (true) {
            ThisThread::sleep_for(1s);
        }
    }
```

## Why Sensor Values Keep Changing Even When the Board is at Rest

When your board is stationary, you might see the accelerometer and gyroscope values jumping around a bit. This is completely normal. Here is why:

1. **Sensor noise** — MEMS sensors always have some random noise in their output.
2. **Temperature** — Even small temperature changes in the room can cause readings to drift.
3. **Digital conversion** — Converting analog signals to digital loses some precision (quantization error).
4. **Gravity** — Your accelerometer is always sensing Earth's gravity (~1g) even when still.
5. **Tiny vibrations** — People walking, AC units running, or a slightly unstable surface all get picked up.

To make readings more stable you could try:

- Adding a low-pass filter to smooth things out
- Calibrating the sensor while it is stationary
- Ignoring changes smaller than a threshold (e.g. less than 0.02g)

### How I2C Works on This Board

The STM32 board communicates with sensors using I2C — a protocol that lets multiple devices share the same two wires.

#### I2C Basics

1. **Shared bus** — All sensors connect to the same two pins (SDA = PB11, SCL = PB10).
2. **Unique addresses** — Each sensor has its own address, like houses on a street:
    - Accelerometer/gyroscope: 0x6A (0xD4/0xD5 for write/read)
    - Humidity/temperature sensor: 0x5F (0xBE/0xBF)
    - Pressure sensor: 0x5D (0xBA/0xBB)
    - Magnetometer: 0x1E (0x3C/0x3D)
    - Distance sensor: 0x29 (0x52/0x53)
3. **Address format** — The 7-bit sensor address is shifted left by 1 bit (`0x6A << 1 = 0xD4`) to make room for the read/write flag in the last bit. Mbed handles this automatically once you pass the shifted address.
4. **I2C transaction sequence** — Every register read follows this sequence:
    - Master sends START + device address + write bit
    - Master sends the register address it wants to read
    - Master sends repeated START + device address + read bit
    - Sensor sends back the data byte
    - Master sends STOP
5. **Selecting a sensor** — The microcontroller sends the target address first; only that sensor responds.
6. **Selecting a register** — After the sensor responds, the microcontroller specifies which register to read or write.

Please refer to the LSM6DSL datasheet on Brightspace for more detail.

## Teleplot

Teleplot is a VS Code extension that plots live serial data as graphs. To send a value, print it over serial in this format:

```
>variable_name:value
```

For example:

```
printf(">acc_x:%.3f\n", acc_x_g);
printf(">gyro_z:%.2f\n", gyro_z_dps);
```

Each variable name becomes its own live graph in the Teleplot window. Make sure each print ends with `\n` and has no spaces around the colon.

## Reference Pages

### User Manual

| Section / Figure | Description | Page No. |
|---|---|---|
| **7.12.4** | Sensor Description | **26** |
| **7.15** | I²C Address | **17** |
| **Figure 22** | Schematic Diagram | **42** |
| **Figure 27** | Sensor Schematic | **47** |

### LSM6DSL Datasheet

| Section | Description | Page Range |
|---|---|---|
| **6.3** | I²C Serial Interface | **38** |
| **8** | Register Mapping | **48–51** |

## Note

There are other sensors on the board (temperature, humidity, pressure, magnetometer, distance) for you to explore as well.